COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S3
DATE PERFORMED:	10/6/26	**DATE SUBMITTED:**	10/6/26
NAME:	Ralf Nicolo B. Abello Rodney C. Domingo	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 11.1 **Basic Algorithm Analysis**

1. PROCEDURE OUTPUT

Assess if two arrays have any common values.

```
bool diff(int *x, int *y){
    for(int i = 0; i < y.length; i++){
        if(search(x, y[i])!= - 1){
            return false;
        }
    }
    return true;
}
```

Performing worst case analysis:

- Let m = the length of array x.
- Let n = the length of array y.
- The loop in diff repeats n times.
- Each call to search requires m comparisons.

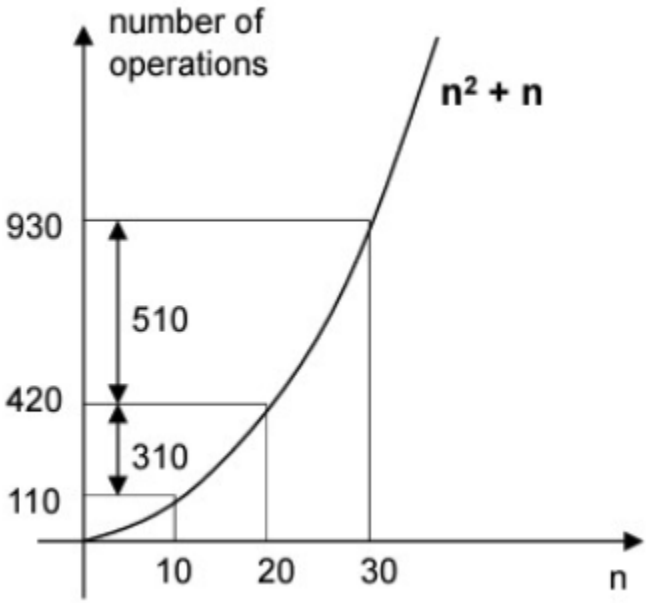
Given the above information, the total number of comparisons in the worst case is: n^2

Explanation:

- Since the outer for loop runs with n times to process each elements in the array y. For every iteration, the search functions checks every array within the variable x with the length of m, which it requires m to compares in the worst case.

- Assume that m = n. (The arrays are the same size)

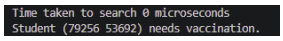
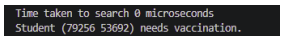
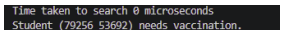
Provide an analysis of the graph for the worse case of the algorithm.



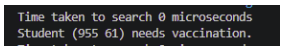
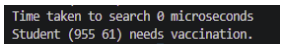
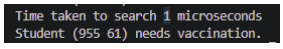
n (size of array)	number of operations
10	110
20	420
30	930

Analysis:
The graph shows a quadratic growth pattern because the number of comparisons increases as the input size increases. Since the outer loop runs **n** times and each search can require **m** comparisons, the total number of comparisons in the worst case is **mn**. When **m = n**, this becomes **n^2**, meaning the algorithm has an **$O(n^2)$** time complexity. Therefore, the running time **increases rapidly** as the size of the arrays becomes larger.

ILO B (ABELLO):

INPUT SIZE (N)	EXECUTION SPEED	SCREENSHOT	OBSERVATION(S)
1000	0 ms		The search took 0 ms, showing that the algorithm was able to find the student quickly with the small input size.
10000	0 ms		The search took 0 ms, showing almost no increase in execution time as the input size became larger.
100000	0 ms		The search took 0 ms again, showing that the search remained very efficient even with a much larger input size.

ILO B (Domingo):

INPUT SIZE (N)	EXECUTION SPEED	SCREENSHOT	OBSERVATION(S)
1000	0 ms		The search took 0 ms, showing high efficiency for the small input size.
10000	0 ms		The search took 0 ms again, showing a continuous high efficiency as the input size increased.
100000	1 ms		The search took 1 microsecond, showing a slight increase in the execution time even as the input size grew significantly.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

For each of the problems below, provide the following:

- Theoretical Analysis
- Experimental Analysis
- Analysis and Comparison

Problem 1: Consider a program that returns true if the elements in an array are unique.

```
Algorithm Unique(A)
  for i = 0 to n-1 do
    for j = i+1 to n-1 do
      if A[i] equals to A[j] then
        return false
  return true
```

CODE

Pair9_ILOB.cpp > Supp1_DOMINGO.cpp > runExperiment(const std::vector<int>&)

```cpp
#include <iostream>
#include <vector>
#include <chrono>
#include <iomanip>

bool isUnique(const std::vector<int>& A) {
    int n = A.size();
    for (int i = 0; i < n - 1; ++i) {
        for (int j = i + 1; j < n; ++j) {
            if (A[i] == A[j]) {
                return false;
            }
        }
    }
    return true;
}

void runExperiment(const std::vector<int>& sizes) {
    std::cout << std::left << std::setw(12) << "Size (n)"
              << std::setw(25) << "Worst-Case Time (ms)"
              << std::setw(25) << "Best-Case Time (ms)" << "\n";
    std::cout << std::string(62, '-') << "\n";

    for (int n : sizes) {
        // Worst Case: All elements unique (must run all comparisons)
        std::vector<int> worstCaseArr(n);
        for (int i = 0; i < n; ++i) worstCaseArr[i] = i;

        auto start = std::chrono::high_resolution_clock::now();
        isUnique(worstCaseArr);
        auto end = std::chrono::high_resolution_clock::now();
        double worstTimeMs = std::chrono::duration<double, std::milli>(end - start).count();
```

```cpp
        // Best Case: Duplicate at the very start (exits on first loop pass)
        std::vector<int> bestCaseArr(n);
        bestCaseArr[0] = 1;
        bestCaseArr[1] = 1;
        for (int i = 2; i < n; ++i) bestCaseArr[i] = i;

        start = std::chrono::high_resolution_clock::now();
        isUnique(bestCaseArr);
        end = std::chrono::high_resolution_clock::now();
        double bestTimeMs = std::chrono::duration<double, std::milli>(end - start).count();

        std::cout << std::left << std::setw(12) << n
                  << std::setw(25) << worstTimeMs
                  << std::setw(25) << bestTimeMs << "\n";
    }
}

int main() {
    std::vector<int> test1 = {1, 5, 8, 3, 9};
    std::vector<int> test2 = {1, 5, 8, 5, 9};

    std::cout << std::boolalpha;

    std::cout << "--- Experimental Analysis ---\n";
    runExperiment({5000, 10000, 20000, 40000});

    return 0;
}
```

Abello:

- Theoretical Analysis: I counted how many times *isUnique* compares two numbers. In the worst case, all the numbers are different, so the function never stops early and checks every pair, which gives **(n-1) + (n-2) + ... + 1 = n(n-1)/2** comparisons. This is **O(n²)**, so doubling n makes the work about 4 times bigger (45 comparisons at n = 10 and 499,500 at n = 1000). In the best case, the first two numbers are the same, so it stops after only 1 comparison, which is **O(1)**, no matter how big the array is.

- Experimental Analysis:

```
--- Experimental Analysis ---
Size (n)    Worst-Case Time (ms)     Best-Case Time (ms)
------------------------------------------------------------
5000        19.7348                  0.0001
10000       79.2464                  0.0001
20000       332.928                  0.0001
40000       1322.41                  0.0001
```

- Analysis and Comparison: In my theoretical analysis, the worst case of *isUnique* does **n(n-1)/2** comparisons, which is **O(n²)**, so doubling **n** should make the work about 4 times bigger. My experiment matches this. The worst-case time went from **19.7348 ms** at **n = 5,000** to **79.2464 ms**, **332.928 ms**, and **1322.41 m**s at **n = 40,000**, which is about 4 times bigger each time n doubled (ratios of 4.02, 4.20, and 3.97). The small differences I noticed might come from the computer, such as the cache and background programs, since the theoretical count is exact. The best case stayed at **0.0001 ms** for every size, which matches the theory that the function stops after only 1 comparison, which is **O(1)**. Both analyses agree that **isUnique** is **O(n²)** in the worst case and **O(1)** in the best case.

Domingo:

- Theoretical Analysis:
  - It evaluates the algorithm by counting the total number of the element comparisons needed across different inputs. The operation performed by the algorithm is the array equality comparison inside the nested loop structure. With the worst case, when all of the elements in the array are considered unique, the loops makes a specific equation to compare, yielding a quadratic time complexity of O(n^2).
- Experimental Analysis:

```
--- Experimental Analysis ---
Size (n)    Worst-Case Time (ms)    Best-Case Time (ms)
------------------------------------------------------------
5000        20.6023                 0.0001
10000       79.2351                 0.0002
20000       383.485                 0.0002
40000       1358.51                 0.0002
```

- Analysis and Comparison:
  - The timing of the experimental side of this algorithm slightly resembles the theoretical worst case complexity, as quadrupling input size n increases the runtime about roughly sixteen folds. With the best case scenario, execution time remains near 0 across all of the array sizes, validating the theoretical analysis.

Problem 2: Consider another program that raises a number x to an arbitrary nonnegative integer, n.

```
Algorithm rpower(int x, int n):
1 if n == 0 return 1
2 else return x*rpower(x, n-1)

Algorithm brpower(int x, int n):
1 if n == 0 return 1
2 if n is odd then
3   y = brpower(x, (n-1)/2)
4   return x*y*y
5 if n is even then
6   y = brpower(x, n/2)
7   return y*y
```

CODE

Pair9_ILOB.cpp > Supp2_DOMINGO.cpp > main()

```cpp
#include <iostream>
#include <chrono>
#include <iomanip>
#include <vector>

long long rpower(int x, int n) {
    if (n == 0) return 1;
    else return x * rpower(x, n - 1);
}

long long brpower(int x, int n) {
    if (n == 0) return 1;
    if (n % 2 != 0) {
        long long y = brpower(x, (n - 1) / 2);
        return x * y * y;
    } else {
        long long y = brpower(x, n / 2);
        return y * y;
    }

}
void runPowerExperiment(const std::vector<int>& exponents) {
    int base = 2;
    std::cout << std::left << std::setw(15) << "Exponent (n)"
              << std::setw(25) << "rpower Time (ns)"
              << std::setw(25) << "brpower Time (ns)" << "\n";
    std::cout << std::string(65, '-') << "\n";

    for (int n : exponents) {
        // Measure rpower
        auto start = std::chrono::high_resolution_clock::now();
        volatile long long res1 = rpower(base, n);
```

```
        auto end = std::chrono::high_resolution_clock::now();
        double rpowerTime = std::chrono::duration<double, std::nano>(end - start).count();

        // Measure brpower
        start = std::chrono::high_resolution_clock::now();
        volatile long long res2 = brpower(base, n);
        end = std::chrono::high_resolution_clock::now();
        double brpowerTime = std::chrono::duration<double, std::nano>(end - start).count();

        std::cout << std::left << std::setw(15) << n
                  << std::setw(25) << rpowerTime
                  << std::setw(25) << brpowerTime << "\n";
    }
}

int main() {
    std::cout << "--- Problem 2 Verification ---\n";
    std::cout << "rpower(2, 10) = " << rpower(2, 10) << "\n";
    std::cout << "brpower(2, 10) = " << brpower(2, 10) << "\n\n";

    std::cout << "--- Experimental Analysis ---\n";
    runPowerExperiment({100, 1000, 5000, 10000, 20000});

    return 0;
}
```

Abello:

- Theoretical Analysis: I counted how many times each function calls itself. *rpower* lowers **n** by only 1 each time, so it makes **n + 1** calls, which is **O(n)**, and doubling **n** doubles the calls. *brpower* cuts **n** in half each time, so it makes only about **log2n + 2** calls, which is **O(log n)**, and doubling **n** adds just 1 more call. At n = 1000, *rpower* makes 1001 calls but *brpower* makes only 11.

  -

- Experimental Analysis:

```
--- Experimental Analysis ---
Exponent (n)    rpower Time (ns)          brpower Time (ns)
-----------------------------------------------------------------
100             900                       100
1000            29000                     200
5000            139200                    100
10000           220800                    200
20000           399000                    200
```

- Analysis and Comparison:

  In my theoretical analysis, *rpower* makes **n + 1** calls, so it is **O(n)**, while *brpower* halves n each time and makes about **log2(n) + 2** calls, so it is **O(log n)**. My experiment agrees with this. The time of *rpower* kept growing, from **900 ns** at **n = 100** to **399,000 ns** at n **= 20,000**, and it became about **1.8 times bigger** when **n** doubled from **10,000** to **20,000**, which is close to linear. The time of brpower stayed at only **100** to **200** ns for every size. The small differences from the theory come from the timer and the computer, since very short times are hard to measure. Both analyses agree that *brpower* is much faster than *rpower* for large **n**.

Domingo:

- Theoretical Analysis:
  - It evaluates the efficiency of calculating x^n by comparing a linear recursion. The operation for both of the algorithms is scalar multiplication performed during the execution phase. In “rpower”, it reduces the n by 1 for each step it requires n recursive calls, resulting in a linear time complexity of O(n) and O(n).
- Experimental Analysis:

```
--- Experimental Analysis ---
Exponent (n)    rpower Time (ns)        brpower Time (ns)
-----------------------------------------------------------------
100             900                     100
1000            28300                   100
5000            145100                  200
10000           198300                  200
20000           397600                  100
```

- Analysis and Comparison:
  - This algorithm aligns directly with the theoretical analysis, showing that the rpower runtime scales linearly O(n) while the brpower remains still. Doubling the n value roughly doubles the execution time for rpower, while brpower only adds a nanoseconds due to the time complexity of O(n log n).

CONCLUSION (ABELLO)
Is this activity, I learned that the running time of an algorithm can be studied in two ways which is theoretical analysis, where I count the operations as a functions of the input size **n**, and experimental analysis, where I measure the real time using **std::chrono**. In the procedure, I saw that the diff functions has a worst case of about **$n^2 + n$** operations, and that the binary search for the student list is much faster than checking every student one by one because it cuts the list in half each time. In the supplementary activity.**Unique(A)** was **O(n^2)** In the worst case and **O(1)** in the best case, and *brpower* was **O(log n)**,while *rpower* was **O(n)**, and my measured times followed these theoretical results closely, with only small differences caused by the timer and the computer. I think I did well in this activity because my theoretical and experimental results agreed, but I still need to improve in explaining my analysis more clearly and in working with algorithms.

CONCLUSION (DOMINGO)
I discovered how to assess algorithm efficiency by measuring the execution time in milliseconds and nanoseconds by using empirical profiling and theoretical operation counting. The analysis of the original process showed that when input sizes are equal, nested searches across arrays produce a m times n comparison structure, which leads to a quadratic O(n^2) growth pattern. Additionally, the supplementary activity empirically confirmed these predictions: brpower demonstrated the enormous advantage of divide-and-conquer O(log n) logarithmic recursion over linear O(n) recursion, while isUnique demonstrated O(n^2) worst-case timing scaling alongside O(1) best-case short-circuiting. In order to reduce system measurement noise during nanosecond-level tests, I intend to concentrate on isolating profiling setup overhead in subsequent lab exercises.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used:

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use:

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.